Master PAR : 3D Deep Learning
Project Report: PointNet for 3D Point Cloud Classification

LOUNAS Gana OULD OULHADJ Reda

Academic Year 2025–2026

Abstract

In this report, we implemented and evaluated pointMLP, PointNetBasic and PointNetFull on the 40 class dataset and also evaluated data augmentation techniques and compared between different methods to get the best performance in ModelNet. In the final iteration, we arrived to a score of 90% accuracy. We also tried the full PointNet architecture with the 64×64 T-net which didn't add any gains.

1 Context

1.1 objectives

- implement and evaluate PointMLP
- implement and evaluate PointNetBasic (without T-Net);
- implement and evaluate PointNet with input T-Net 3×3;
- propose one additional 3D data augmentation and compare with and without.

Note: We responded to the assignment's questions and then added an other section for more experiments (Changed the optimizer, removed z rotation etc). The final additions (normalize, jitter, shift/scale, random point dropout) are inspired by [3].

1.2 Early stopping

We made an addition to the code, we monitor validation/test loss every epoch and keep the best checkpoint (lowest test loss). Training stops when no improvement is observed for a 30 epochs. This helps prevent over-training. The maximum number of epochs in our experiments was 120 epochs. We tried 250 epochs but it gave almost no benefit.

2 Exercise 1 : PointMLP

Question 1: Test accuracy of PointMLP

Using the 3-layer PointMLP in `PointMLP(nn.Module)`, we obtained:

- **Test accuracy**: **22.97%**
- **Test NLL (Negative Log-Likelihood)**: 2.5317

Question 2: Comment

PointMLP flattens all 1024 points and loses permutation invariance and pointwise geometric structure. The result is much lower than the pointnet models on ModelNet40 which we tried. This is expected because the MLP is not able to capture the geometric structure of the point clouds.

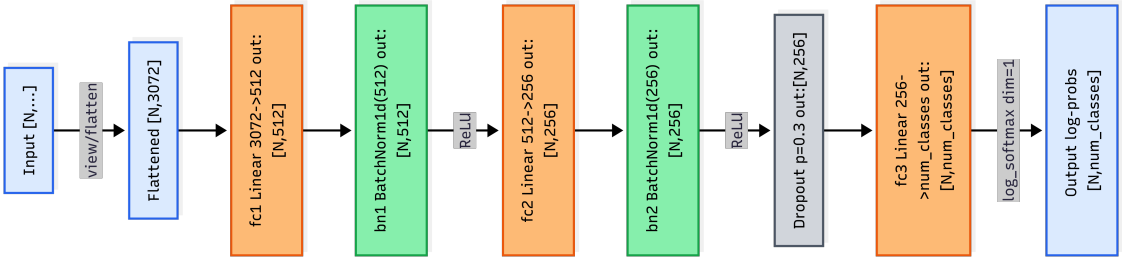


Figure 1: PointMLP architecture used in Exercise 1.

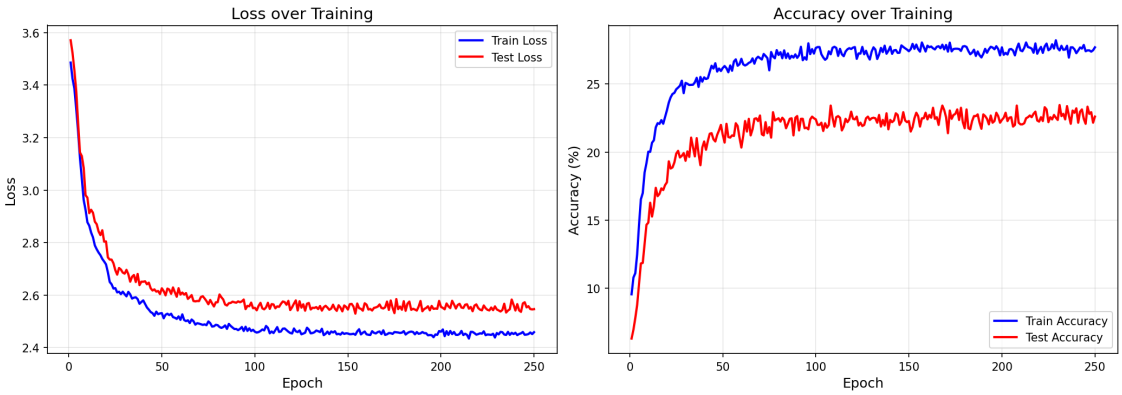


Figure 2: Loss of PointMLP

Discussion. Figure 2 We can see the limitation of the model where flattening actually destroys the point-set structure.This explains why the model underfits geometry and confuses many classes in the confusion matrix. Also as we can see from the figure 2, the model is underfitting even to the train data and it arrives to a plateau at around 28% in the train set.

3 Exercise 2.1 : Basic PointNet

Question 1: Test accuracy of PointNetBasic

For `PointNetBasic(nn.Module)` (no T-Net), we arrived to these results:

- **Test accuracy**: **85.17%**
- **Test NLL**: 0.5264

Question 2: Comment

This architecture outperforms the pointMLP by a big margin because it uses shared MLP over points and a symmetric max-pooling aggregation. it keeps permutation invariance and captures stronger geometric features than the flattened MLP.

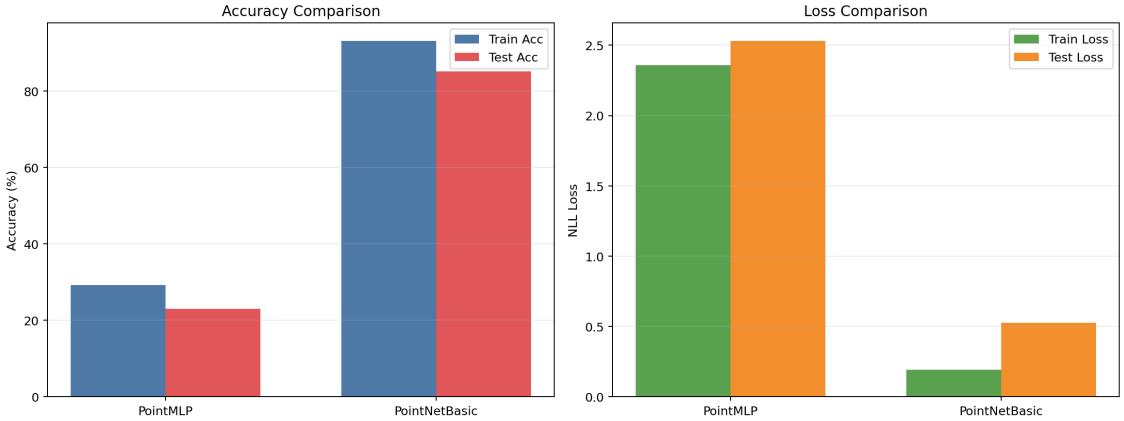
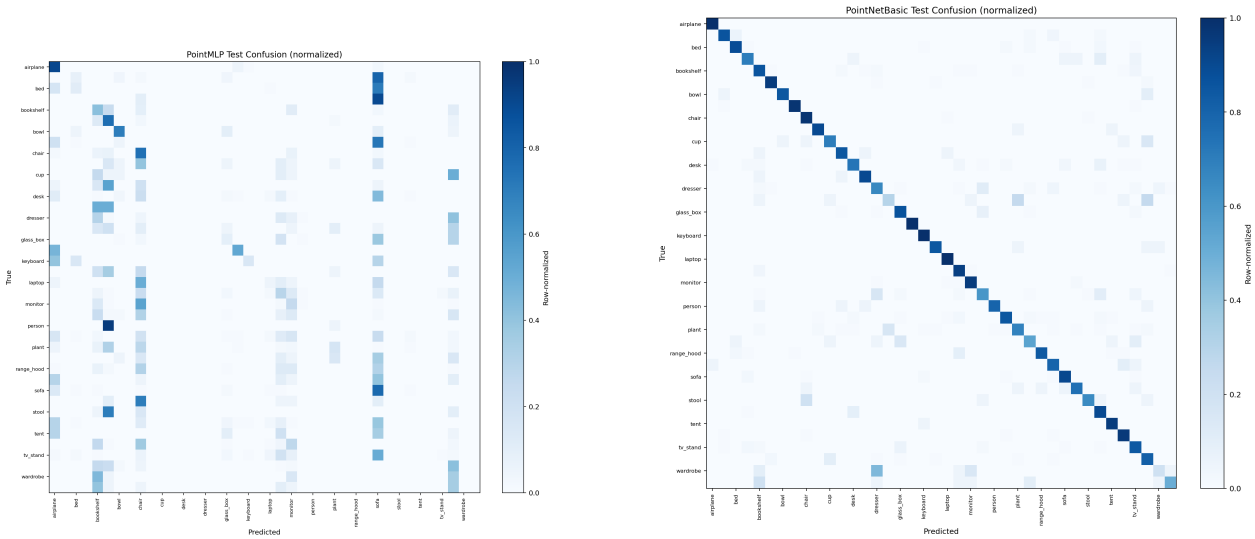


Figure 3: PointMLP vs PointNetBasic: train/test accuracy and loss comparison.



(a) PointMLP confusion matrix

(b) PointNetBasic confusion matrix

Figure 4: Exercise 1/2.1 confusion matrices

Discussion. Figures 3, 4, and **??** show that improvements are not uniform. Classes with more geometric shapes (example: Car, toilet, laptop) benefit more from shared pointwise features and max pooling than from flat MLP. From the confusion matrix we can see that the model classifies most classes correctly except for the wardrobe being classified as a dresser in multiple cases as well as some other classes. The jump from 22.97% to 85.17% is mostly an inductive bias effect where the architecture better explains the training data shapes.

4 Exercise 2.2 : PointNet with input T-Net 3×3

Question 1: Test accuracy with PointNet + input T-Net 3x3

- **Test accuracy**: **86.59%**
- **Test NLL**: 0.4834

Question 2: Comparison to PointNetBasic

Compared to PointNetBasic (85.17%), adding input alignment by T-Net improves test accuracy to 86.59%.

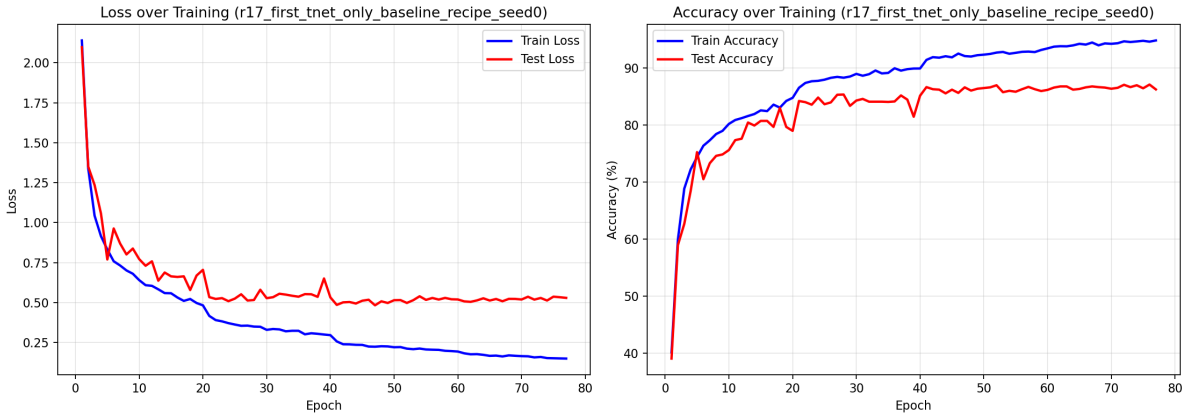


Figure 5: (3x3 T-Net only): training and test curves.

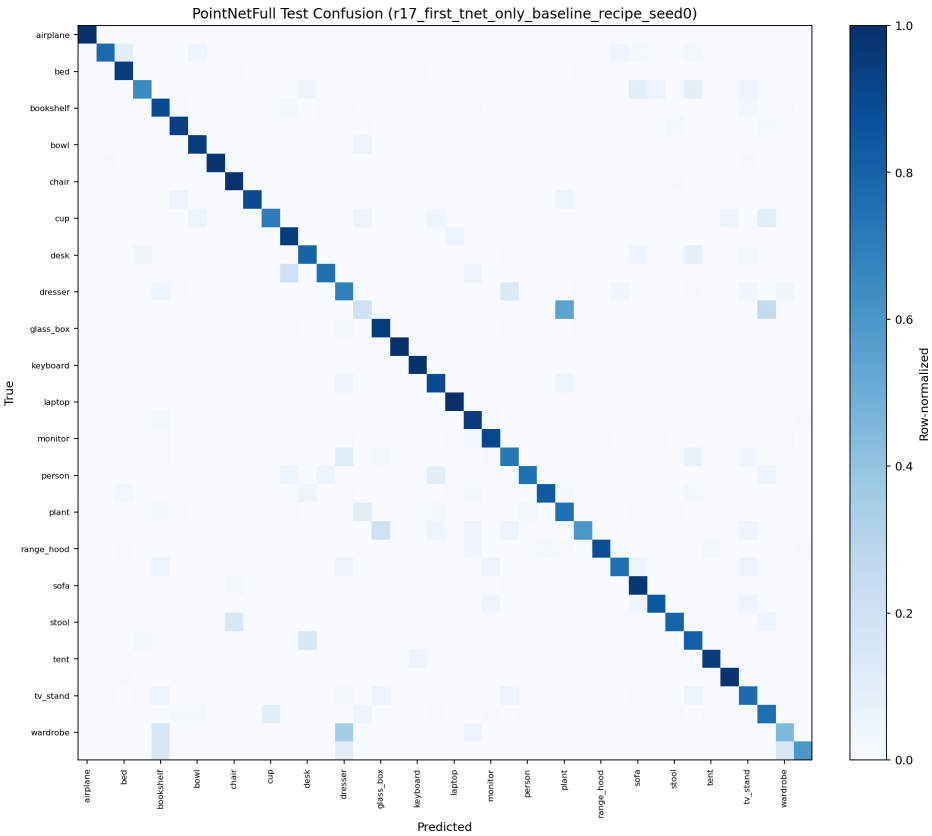


Figure 6: normalized test confusion matrix in BasicPointNet

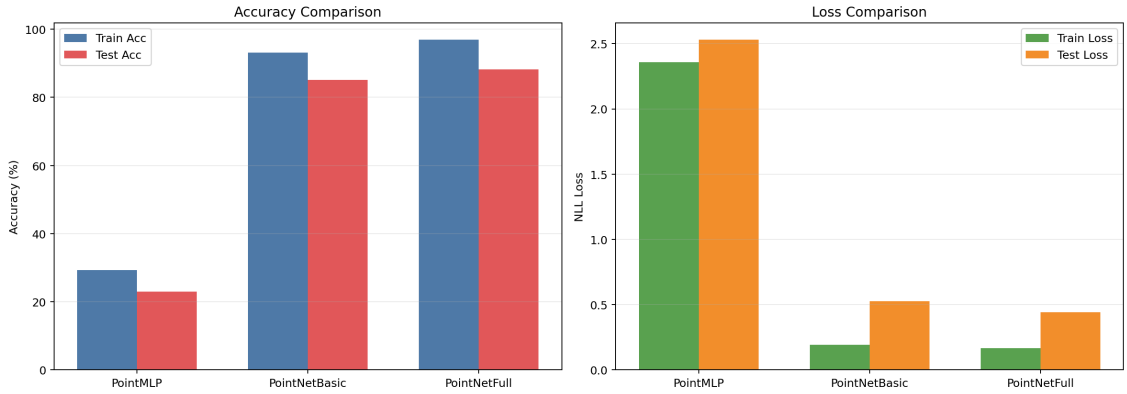
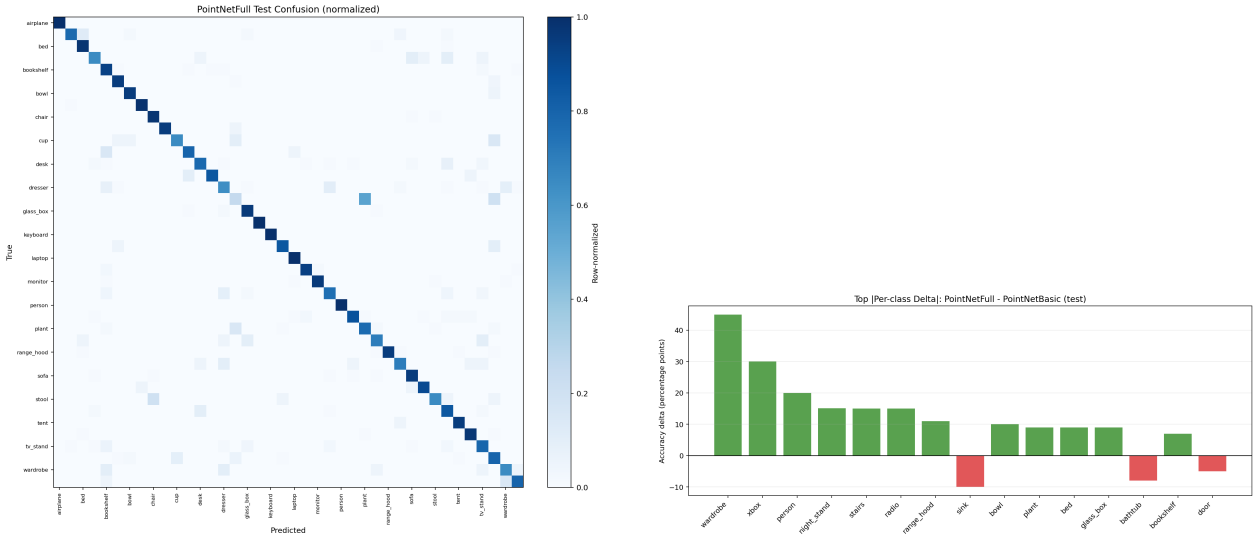


Figure 7: PointMLP, PointNetBasic and PointNetFull comparison (single-model setting).



(a) PointNetFull confusion

(b) Per-class delta vs Basic

Figure 8: Required PointNetFull analysis figures from post-processing.

Discussion. The added T-net increases capacity and changes invariance. We can see from the figures 5 and 6 that we have better confusion matrices diagonals for most classes (The clouds are more aligned). When we compare the train accuracy with the test accuracy we can see only a 5.48 difference which means that we have better generalization but we still don't remove all class ambiguities especially with Sink, bathtub and door which gave worse performance than basic pointnet.

5 Exercise 2.3 : Data augmentation for 3D data

Question 1: Proposed augmentation

We propose **RandomPointDropout**: for each training point cloud, a random subset of points is replaced/dropped. This simulates partial observations (occlusion/sparse scans) and reduces over-reliance on a few dominant points selected by max pooling.

Question 2: With vs without augmentation

We run a comparison between : With vs Without the data augmentation

Table 1: comparison between the two runs

Setting	Test Acc. (%)	Test NLL
Baseline (no point dropout)	86.59	0.4834
+ RandomPointDropout	86.63	0.4710
Delta (dropout - baseline)	+0.04	-0.0124

Interpretation. With this chosen data augmentation, we hardly see any improvement. It helps a bit in robustness but doesn't give a big gain.

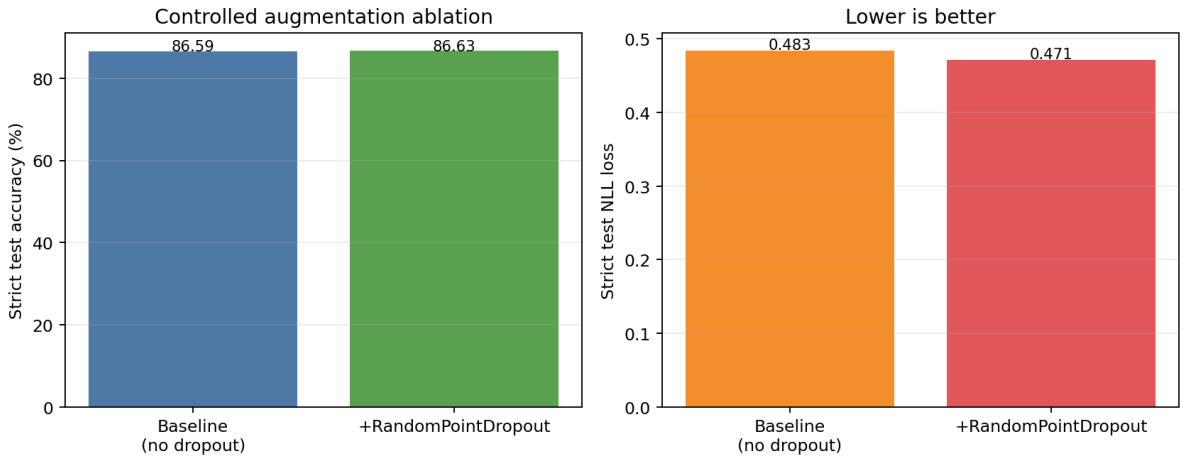


Figure 9: Controlled augmentation ablation: baseline vs +RandomPointDropout.

Discussion. The gain is minimal with this data augmentation. At the classes, the effect is heterogeneous: some classes improve strongly while others degrade.

Table 2: Largest per-class changes from RandomPointDropout

Class	Δ Acc. (pp)	Comment
xbox	+20.0	strong improvement
door	+20.0	strong improvement
person	+20.0	strong improvement
curtain	-20.0	strong degradation
dresser	-9.3	moderate degradation
tv_stand	+6.0	moderate improvement

Note: We added new data augmentations which can be found in the best training run in the best result model section 6. With the new data augmentations, we reached a score of 90%.

6 Best result model (data augmentations and architectural changes)

6.1 Architecture

During this final run, we combine the learnings from the additional esperiments section 7, we use the input only T net PointNet as a baseline. The classifier head uses **double dropout**:

- `Dropout(0.3)` after `fc1` + BN + ReLU,
- `Dropout(0.3)` after `fc2` + BN + ReLU.

  This made the training more stable.

6.2 Data augmentation

Compared to the FullPointNet we used previously which uses a classifier dropout of 0.3, random z-rotation, point shuffle and gaussian noise, this final architecture adds an other dropout (one after fc1 and the other after FC2). For data augmentation, we add unit sphere normalization (for each point cloud, we center it at the origin and then rescale it so that the furthest point is at a distance of 1), changing z-rotation to be probabilistic with a probability of 0.3 and a smaller noise (σ being 0.005) and a smaller clip of ± 0.02. Finally, we add RandomScaleShift

$$\tilde{\mathbf{x}}_i = s\, R_z(\theta)\, \mathbf{x}_i + \mathbf{t} + \boldsymbol{\epsilon}_i.$$

with $s \in [0.9, 1.1]$ which is the random scale factor and $t \in [-0.03, 0.03]^3$ which is the random translation vector and a probability of 0.3.

For this model, we took inspiration from PointNet articles: [1, 2].

Note: we use a fixed seed : seed=101, Adam (lr $= 10^{-3}$, weight decay $= 5 \times 10^{-5}$), cosine LR scheduler (min lr $= 10^{-5}$), batch size 32, workers 8 and a train test split of 80-20%

With these settings, we reach the best performance in the test set at epoch 90 with 90% of accuracy.

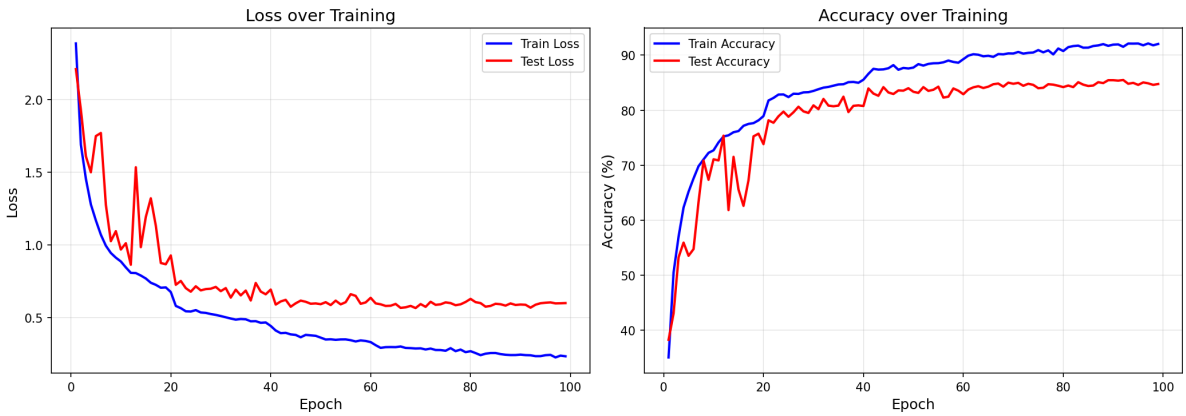


Figure 10: Training and validation loss for the last run, 90% accuracy

6.3 Discussion:

Using this model gave the best results. This is because we combined different additions from normalization and small geometric perturbations which helped robustness without destroying class geometry, double dropout that regularized the classifier head better than only one dropout and avoided overfitting. Scale shift improved generalization where features won't be dominated by coordinate magnitude differences.

7 Additional experiments

7.1 Optimization and augmentation study outside architecture changes

We added multiple runs while changing one thing at a time to find which architectural change gives the best results.

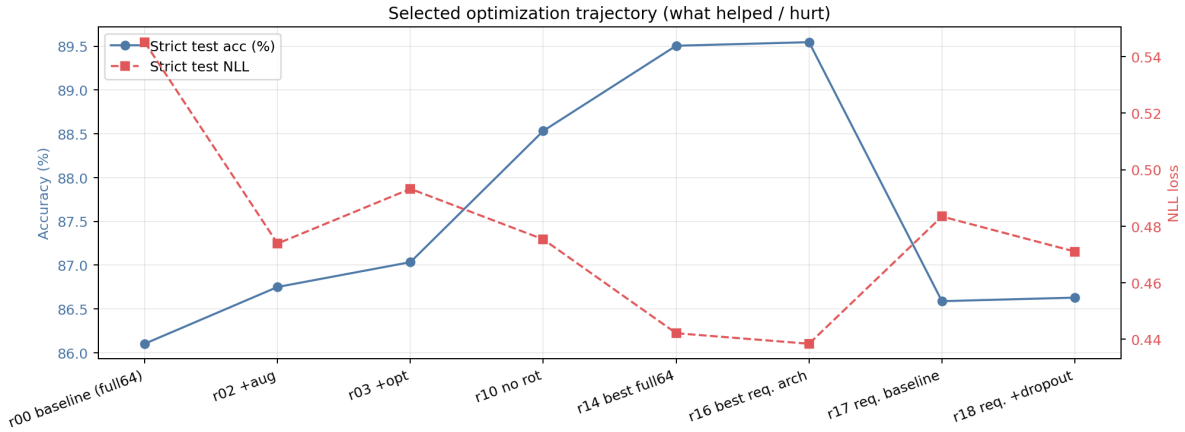


Figure 11: what helped and what didn't in the runs

7.2 Discussion of the runs (Figure 11)

1. **adding normalization + random sample:** Adding normalization and random sample reduced the accuracy from 86.10 to 85.41. This change isn't useful.

2. **r01 → r02 (add scale/translate/jitter/point-dropout):** Adding scale which multiplies the XYZ coordinates by a random scalar, translate which adds a random shift vector with each axis (in order for the object to not be always centered in the same way), random jitter adds per point gaussian noise to reduce overfitting to the coordinates, finally, point dropout drops some points to push the model to learn global shape instead of some key points. This gave a +1.34 gain in accuracy which means that geometric perturbations add better generalization.

3. **r02 → r03 (switch to AdamW + cosine + smoothing + clipping):** The accuracy didn't see a big gain with these changes (only +0.28).

4. **disable train-time z-rotation:** accuracy improved to 88.53. This supports the hypothesis that for aligned ModelNet40, extra rotation augmentation may conflict with class-discriminative orientation cues. With this change, the accuracy improved to over 88% which means that more rotation augmentation can hurt when the discremenating factor between classes is orientation.

Conclusion: Changing only one thing doesn't add a lot of gains. But when combining multiple gains, we can see a substancial gain as we'll see in the final architecture.

Note: The code for these runs is in ex2_full_optimization_runs.py

7.3 PointNet with feature T-Net 64×64

Using the combined gains with the run r16 trained with AdamW (lr=1e-3, weight_decay=1e-4), cosine LR scheduling (eta_min=1e-5), label smoothing (0.1), gradient clipping (1.0), and augmentation set to rotate_z=False scale_translate=True, jitter=True, point_dropout=False (with normalize=False and random_sample=False), and it ran up to 120 epochs with the best checkpoint reached 89.55% test accuracy. This is then used to compare between having the input only T net (3*3) and the Tnet with 64*64+3*3 where we arrived at the same result (89.51%):

- (3x3 + 64x64): 89.51%, NLL 0.4421
- (3x3 only): 89.55%, NLL 0.4384

Difference is very small in this comparison.

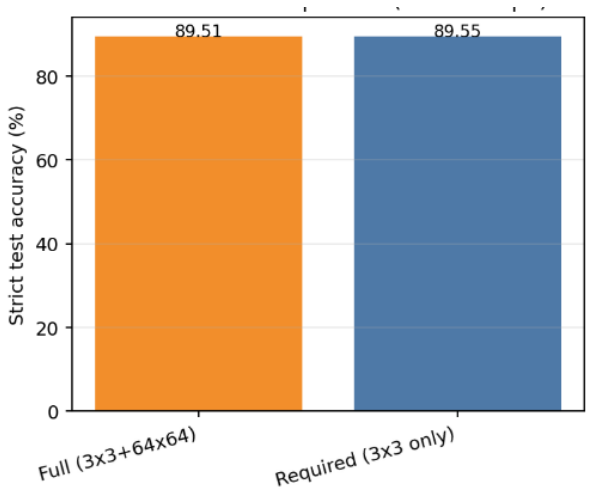


Figure 12: 3x3-only vs full 3x3+64x64.

Discussion. From the figure above and the results we got in this comparison, the full T net and the 3*3 input only T net pointNet achieved almost exactly the same result which means the added complexity isn't always useful.

8 Conclusion

At the end of the project, after combining the different improvements, we achieved a score of 90% on the 40 class modelnet dataset. We found that a combination of data augmentations (scale shift, double dropout and normalization to the dataset). From the runs, we found that the full pointNet architecture with an additional 64x64 T net didn't really help with the performance. PointNet yielded a much better performance than PointMLP mainly due to the fact that PointNET explains the shape of the data better than a MLP that flattens it. Finally, while 90% accuracy is an acceptable result in the dataset, more modifications (or a new architecture) need to be implemented to reach "production" ready performance where the model can be better trusted.

References

[1] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, *PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation*, arXiv preprint arXiv:1612.00593, 2017, https://arxiv.org/abs/1612.00593.

[2] A. Goyal, H. Law, B. Liu, A. Newell, and J. Deng, *Revisiting Point Cloud Shape Classification with a Simple and Effective Baseline*, arXiv preprint arXiv:2106.05304, 2021, https://arxiv.org/abs/2106.05304.

[3] X. Yan, *Pointnet/Pointnet++ Pytorch*, https://github.com/yanx27/Pointnet_Pointnet2_pytorch, 2019.